TASTES2PLATE · API INTEGRATION GUIDE

# T2P Logistics App: Expenses & Cash Ledger API

Integration guide for the logistics app developer

**Contents**


1. Overview
2. Authentication
3. Conventions (read before coding)
4. How an expense works
5. Data needed when the app starts
6. The expense categories
7. Working with expenses
8. My cash: `GET /v1/cash-ledger/me`
9. Push notifications
10. Endpoint reference (app)
11. Testing checklist
12. Current limitations


This guide covers everything the logistics app needs to let delivery partners, delivery boys, pickup partners and other logistics users:

- record their expenses,
- send them for approval,
- see their own cash balance.

The office (admin panel) approves or rejects expenses and records cash given by Head Office. The app never does those things.

## 1. Overview

### 1.1 What the app needs to build

| Screen | What it does | Main API calls |
|---|---|---|
| Expenses home | My expenses with status tabs (Draft, Pending approval, Approved, Rejected, Cancelled) and my cash balance | `GET /v1/expenses`, `GET /v1/cash-ledger/me` |
| New expense: choose type | Seven categories | `GET /v1/expenses/meta` |
| Food Purchase form | Tick orders, then enter rate, weight and discount for each product, with quantity changes if needed | `food-purchase-orders`, `food-purchase-lines`, `POST /v1/expenses` |
| Intercity Logistics form | From city (locked), To city, rate per kg, weight | `cities`, `POST /v1/expenses` |
| Delivery form | Order, default distance (shown, read-only), actual distance | `lookups/orders`, `default-distance`, `POST /v1/expenses` |

| Screen | What it does | Main API calls |
|---|---|---|
| Miscellaneous form | Expense head or Other, details, reason, amount | `expense-heads`, `POST /v1/expenses` |
| Petrol form | Rate per litre, litres, start km reading, vehicle, pump | `POST /v1/expenses` |
| Travelling form | From, to, travel mode, date, purpose, amount | `POST /v1/expenses` |
| Cancelled Orders | What happened to the food of a cancelled order: adjusted on another order, consumed or scrapped; partner approves it | `lookups/cancelled-orders`, `POST /v1/expenses`, `approve` |
| Expense detail | Details, photos, history, rejection reason; Edit, Submit, Move to draft, Cancel buttons | `GET /v1/expenses/:id`, actions |
| My cash | Balance, period summary, list of entries | `GET /v1/cash-ledger/me` |

### 1.2 Base URL

| Environment | Base URL |
|---|---|
| Production | `https://webapi.tastes2plate.com` (available once T2P deploys the expense module) |
| Test | Ask T2P for the test server URL |

Every expense endpoint starts with `/v1`. The login endpoints are the existing `/app/...` ones.

### 1.3 Who can use it

Users with `user_type` delivery_boy, delivery_partner, pickup_boy, pickup_partner, cargo_partner, lp_head or lp_manager. Each user only ever sees **their own** expenses and cash ledger.

## 2. Authentication

### 2.1 Login (existing OTP flow, now returns a token)

**Step 1: request OTP** (unchanged)

```
POST /app/logistic-login
Content-Type: application/json

{ "mobile": "9800000010" }
```

> **Note:** this response currently also contains the OTP. Do **not** use it. It will be removed; always ask the user to type the OTP they received by SMS.

**Step 2: verify OTP and get the token**

```
POST /app/verify-logistic
Content-Type: application/json
```

```
{ "mobile": "9800000010", "otp": "482913" }
```

```
{
  "status": "success",
  "message": "Otp verified",
  "result": {
    "_id": "605dc565d5eaba0008a50e4e",
    "full_name": "Ravi Kumar",
    "user_type": "delivery_partner",
    "...": "..."
  },
  "office_data": { "_id": "60ef0e96d4473c18f88872f5", "name": "KOLKATA OFFICE", "...": "..." },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...."
}
```

Only `token` is new; the other fields are the same as before.

### 2.2 Using the token

- Save `token` in secure storage (Keychain on iOS, EncryptedSharedPreferences/Keystore on Android).
- Send it on **every** `/v1` request:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
```

- The token is valid for **30 days**.
- On HTTP **401**, clear the token and show the OTP login again.
- The server identifies the user from the token. **Never send a user id** to say who the user is; it is ignored or refused.

```
// 401 example
{ "status": "error", "message": "Login required" }
```

## 3. Conventions (read before coding)

### 3.1 Money

| Direction | Format | Example |
|---|---|---|
| **App → server** | Rupees, as a string or number, at most 2 decimals | `"250.50"`, `"80"` |
| **Server → app** | **Paise**, as an integer | `25050` means ₹250.50 |

Display rule: `rupees = paise / 100`, shown with 2 decimals.

**The server calculates every total.** Anything you calculate on the phone is only a preview; always show the server's numbers after saving.

## 3.2 Quantities and precision

| Value | Unit | Decimals allowed |
|---|---|---|
| Food quantity, weight | count, kg | 3 |
| Litres, distance | litres, km | 2 |
| Start km reading | km | 1 |
| Money | rupees | 2 |

Negative numbers are refused everywhere, except the signed quantity in Food Purchase adjustments (section 6.1).

## 3.3 Dates

- Send dates as `"YYYY-MM-DD"` (a calendar day in India time), or a full ISO date-time such as `"2026-10-10T09:30:00+05:30"`.
- No future dates.
- The app can backdate at most **7 days** (`meta.backdate_days`).
- If you leave `expense_date` out, it is "now".

## 3.4 Response format

Every response is JSON with this envelope:

```
{ "status": "success", "message": "Expense saved", "data": {} }
```

| HTTP | `status` | Meaning | What the app should do |
|---|---|---|---|
| 200 / 201 | `success` | OK | Use `data` |
| 401 | `error` | Not logged in or token expired | Go to OTP login |
| 403 | `error` | Not allowed (e.g. the order isn't assigned to this user) | Show `message` |
| 404 | `error` | Not found, or it belongs to someone else | Show `message`, go back |
| 409 | `error` | Conflict: already exists, or changed since you loaded it | Show `message`, reload the record |
| 422 | `validation_error` | Some fields are wrong; see `errors[]` | Show each error next to its field |
| 429 | `error` | Too many requests (more than 60 changes per minute) | Wait and retry |
| 500 | `error` | Server problem | Show "Try again later" |

**Validation errors:**

```
{
  "status": "validation_error",
  "message": "Please correct the highlighted fields",
```

```
  "errors": [
    {
      "field": "details.rate_per_litre",
      "message": "rate_per_litre must be a positive number with at most 2 decimal places"
    },
    {
      "field": "details.litres",
      "message": "litres must be a positive number with at most 2 decimal places"
    },
    { "field": "details.start_km", "message": "start_km is required" }
  ]
}
```

`field` uses the same path as your request body. Category fields are always under `details.`, for example `details.items[1].rate` or `details.items[0].adjustments[0].note`. Other fields: `expense_date`, `files`, `category`, `reason`.

### 3.5 Safe retries (important on mobile networks)

When creating an expense, generate a random **`client_request_id`** (a UUID) **once per new expense form** and send it with the create request.

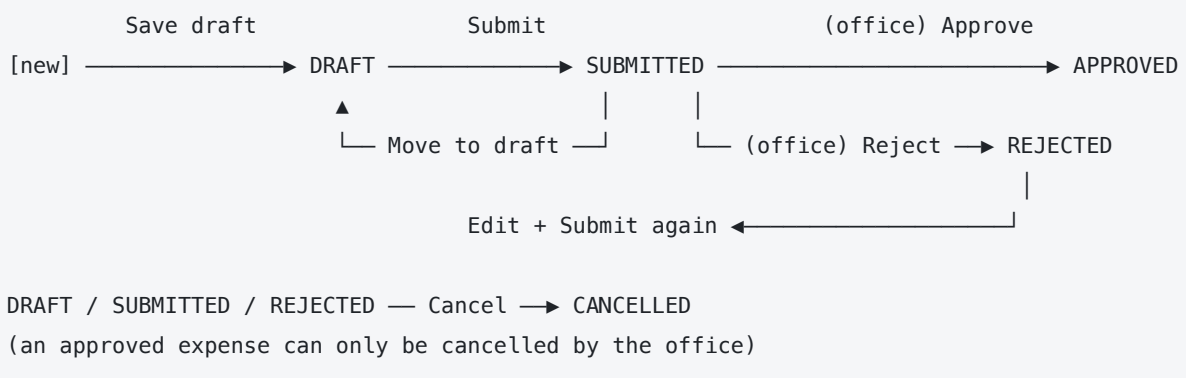
- If the connection drops and you send the same request again with the same id, the server returns the **already saved** expense (HTTP 200, message "This expense was already saved") instead of creating a duplicate.
- Generate a new id only when the user starts a new expense.

### 3.6 Versions (optional for the app)

Every expense has a `version` number that goes up on each change. You may send `"version"` with edit, submit, move-to-draft and cancel. If someone else changed the expense in between, you get **409**; reload and show the fresh data. If you don't send it, the latest version is used.

## 4. How an expense works

### 4.1 Status flow



| Status | Shown to user as | User can |
|---|---|---|
| `draft` | Draft | Edit, add or remove photos, Submit, Cancel |
| `submitted` | Pending approval | Move to draft (to edit), Cancel |
| `approved` | Approved | Only view |
| `rejected` | Rejected (show `review.rejection_reason`) | Edit, add or remove photos, Submit again, Cancel |
| `cancelled` | Cancelled | Only view |

**Cancelled Orders are different:** there is no Submit and no office approval. The partner/boy saves a draft, can edit it, and taps **Approve** when the record is complete (`POST /v1/expenses/:id/approve`). Approved is final. See section 6.7.

**Don't hard-code these rules.** Every expense response contains an `actions` object; show a button only when its value is `true`:

```
"actions": {
  "edit": true, "upload": true, "submit": true, "recall": false,
  "approve": false, "reject": false, "cancel": true
}
```

(`approve` and `reject` are always `false` for app users.)

### 4.2 Effect on the user's cash

| What happens | Cash ledger |
|---|---|
| Save as draft | No change |
| **Submit** (or submit again) | **Amount is deducted immediately** (entry type `EXPENSE_DEBIT`) |
| Office approves | No change (it was already deducted) |
| Office rejects / user cancels / user moves it back to draft | **Amount is put back** (`EXPENSE_REVERSAL`) |

The submit response includes the new balance (`balance_after`, in paise). It also includes a `warning` if the balance is now negative; show that warning to the user.

## 5. Data needed when the app starts

### 5.1 `GET /v1/expenses/meta`

Call it when the user opens the Expenses section. Cache it for the session.

```
{
  "status": "success",
  "data": {
    "me": {
      "_id": "605dc565d5eaba0008a50e4e",
      "name": "Ravi Kumar",
```

```
    "mobile": "9800000010",
    "user_type": "delivery_partner",
    "is_staff": false,
    "cities": [ "6040eae14a4b6c0008fe1aff" ],
    "default_city": "6040eae14a4b6c0008fe1aff",
    "default_office": "60ef0e96d4473c18f88872f5"
  },
  "permissions": {
    "expense": { "add_edit": true, "approve": false, "...": "..." },
    "ledger": { "...": "..." }
  },
  "categories": [
    {
      "code": "FOOD_PURCHASE",
      "label": "Food Purchase",
      "document_required": false,
      "fields": [ "..." ]
    },
    {
      "code": "INTERCITY_LOGISTICS",
      "label": "Intercity Logistics",
      "document_required": false,
      "fields": [ "..." ]
    },
    { "code": "DELIVERY", "label": "Delivery", "...": "..." },
    { "code": "MISC", "label": "Miscellaneous", "...": "..." },
    { "code": "PETROL", "label": "Petrol Filling", "...": "..." },
    { "code": "TRAVEL", "label": "Travelling", "...": "..." }
  ],
  "statuses": [ "draft", "submitted", "approved", "rejected", "cancelled" ],
  "travel_modes": [
    { "value": "bus", "label": "Bus" },
    { "value": "train", "label": "Train" },
    { "value": "taxi", "label": "Taxi" },
    { "value": "auto_rickshaw", "label": "Auto-rickshaw" },
    { "value": "bike", "label": "Bike" },
    { "value": "flight", "label": "Flight" },
    { "value": "other", "label": "Other" }
  ],
  "expense_heads": [ { "_id": "6ac9...01", "name": "Packaging material", "active": true } ],
  "gst_rate_percent": 5,
  "backdate_days": 7,
  "upload": { "max_files": 5, "max_mb": 10, "allowed": "JPG, PNG, WEBP, HEIC or PDF" },
  "ledger_types": {
    "OPENING_BALANCE": "Opening Balance",
    "CASH_RECEIVED_HO": "Cash Received from Head Office",
    "EXPENSE_DEBIT": "Expense",
    "EXPENSE_REVERSAL": "Expense Reversal",
    "ADJUSTMENT_CREDIT": "Manual Credit Adjustment",
```

```
      "ADJUSTMENT_DEBIT": "Manual Debit Adjustment"
    }
  }
}
```

Use `categories[].label`, `travel_modes`, `expense_heads`, `gst_rate_percent`, `backdate_days` and `upload` from here instead of hard-coding them; the office can change them without an app release.

### 5.2 Other lists

| Call | Returns |
|---|---|
| `GET /v1/expenses/lookups/cities` | `[{ "_id", "name", "ps" }]`, all active cities |
| `GET /v1/expenses/lookups/expense-heads` | `[{ "_id", "name", "active" }]`, heads for Miscellaneous |

## 6. The expense categories

All of them are created with the same call, `POST /v1/expenses`; only `category` and `details` differ (section 7.1). The examples below show only the `details` object.

### 6.1 Food Purchase ( `FOOD_PURCHASE` )

The pickup partner/boy buys food at the restaurant for one or more orders.

**Rules**

- **Which orders:** only orders where this user is the order's **pickup partner or pickup boy**, from the last 30 days, that are not already in another food purchase.
- **Several orders in one purchase:** allowed (up to 50), but all must be from the **same city**.
- **Same product, one line:** if a product is in several orders, it appears **once**, with the quantities added up. Example: order A has Chicken Biryani ×2 and order B has Chicken Biryani ×3, so the line is Chicken Biryani with order quantity **5**.
- **Quantity can't be edited:** the **order quantity is fixed** and comes from the orders. **Don't send a quantity**; the server ignores it.
- **Changing the quantity:** if the user bought more or fewer, they add a **quantity change** ( `adjustments[]` ) with:
  - a signed quantity ( `"+2"` or `"−1"` ),
  - an optional Order ID it is for,
  - a **required reason** ( `note` ).

  Final quantity = order quantity + all changes.
- **Products can't be removed:** if a product wasn't bought, reduce it to 0 with a change and a reason. Then no rate is needed.
- **One purchase per order:** an order can be in only one active food purchase. Cancelling the purchase frees its orders.

- **Totals:** for each product, Subtotal = rate × final quantity; Taxable = subtotal – discount; GST = 5% of taxable; Total = taxable + GST.

**Screen flow**

1. `GET /v1/expenses/lookups/food-purchase-orders?q=T2P-179` shows the list of orders with checkboxes. `q` (search by order number) is optional.


```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac33f67cd32c8f0c6301fa6",
      "orderid": "T2P-1790000101",
      "status": "vendor_approved",
      "created_date": "2026-10-09T11:10:22.917Z",
      "delivery_date": "2026-10-11T00:00:00.000Z",
      "vendor_city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "order_city": { "_id": "608f769984eaa700091774fe", "name": "PUNE" },
      "products": [
        {
          "line_id": "6ac33f67cd32c8f0c6301fa7",
          "product": "65ec39202df3d12ed065251f",
          "product_name": "Chicken Biryani - Example Restaurant, Kolkata",
          "quantity": 2,
          "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" },
          "unit_weight_kg": 0.55
        }
      ]
    }
  ]
}
```


2. When the user has ticked orders, call **`GET /v1/expenses/lookups/food-purchase-lines?orders=<id1>,<id2>`**. It returns one line per product, already combined. **Show the lines in this order**: validation errors refer to `details.items[i]` by this index.


```
{
  "status": "success",
  "data": {
    "orders": [
      {
        "_id": "6ac33f67cd32c8f0c6301fa6",
        "orderid": "T2P-1790000101",
        "status": "vendor_approved"
      },
      {
        "_id": "6ac25d53cd32c8f0c62d7a74",
        "orderid": "T2P-1790000102",
        "status": "vendor_approved"
```

```
      }
    ],
    "vendor_city": "6040eae14a4b6c0008fe1aff",
    "lines": [
      {
        "product": "65ec39202df3d12ed065251f",
        "product_name": "Chicken Biryani – Example Restaurant, Kolkata",
        "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" },
        "orders": [
          { "order": "6ac33f67cd32c8f0c6301fa6", "orderid": "T2P-1790000101", "quantity": 2 },
          { "order": "6ac25d53cd32c8f0c62d7a74", "orderid": "T2P-1790000102", "quantity": 3 }
        ],
        "order_quantity": 5,
        "unit_weight_kg": 0.55
      },
      {
        "product": "60b9b6f57e8bd40009624507",
        "product_name": "Rasgulla 1 kg – Example Sweets, Kolkata",
        "vendor": { "_id": "60b895375df2b80008ba4f8e", "name": "Example Sweets" },
        "orders": [
          { "order": "6ac25d53cd32c8f0c62d7a74", "orderid": "T2P-1790000102", "quantity": 1 }
        ],
        "order_quantity": 1,
        "unit_weight_kg": 1
      }
    ]
  }
}
```

3. **For each line, show:**
    - product name, restaurant, and the orders it came from;
    - **Order quantity** (read-only);
    - a **"Change quantity"** button that opens Add/Less, quantity, Order ID (optional) and Reason (required);
    - the **Final quantity**, recalculated as changes are added;
    - inputs for **Rate**, **Weight (kg)** (you may pre-fill `unit_weight_kg × order_quantity`) and **Discount** (₹ or %).
4. **Request `details`:**

```
{
  "orders": [ "6ac33f67cd32c8f0c6301fa6", "6ac25d53cd32c8f0c62d7a74" ],
  "items": [
    {
      "product": "65ec39202df3d12ed065251f",
      "rate": "250",
      "weight_kg": "2.2",
      "discount_type": "amount",
      "discount_value": "20",
```

```
      "adjustments": [
        {
          "quantity": "-1",
          "orderid": "T2P-1790000102",
          "note": "Only 4 available at the shop"
        }
      ]
    },
    { "product": "60b9b6f57e8bd40009624507", "rate": "180", "weight_kg": "1" }
  ]
}
```

| Field | Required | Notes |
|---|---|---|
| `orders` | yes | Ids of the ticked orders (1–50, same city) |
| `items[]` | yes | **One item for every line** returned by `food-purchase-lines` |
| `items[].product` | yes | From the line |
| `items[].rate` | yes, if final quantity > 0 | Rupees per unit |
| `items[].weight_kg` | no | Total weight for this line |
| `items[].discount_type` | no | `"amount"` (default) or `"percent"` |
| `items[].discount_value` | no | Rupees for `amount`, percent for `percent`; cannot exceed the subtotal |
| `items[].adjustments[]` | no | Up to 20 per line |
| `adjustments[].quantity` | yes | Signed, not 0: `"+2"`, `"-1"`, `"-0.5"` |
| `adjustments[].orderid` | no | An existing order number such as `"T2P-1790000102"` |
| `adjustments[].note` | yes | At least 3 characters: why the quantity changed |

**Result:** Chicken Biryani has order quantity 5 and a change of −1, so the final quantity is 4.

- Subtotal ₹1,000.00, discount ₹20.00, taxable ₹980.00, GST ₹49.00, **total ₹1,029.00**.
- Rasgulla: total ₹189.00.
- **Expense total ₹1,218.00** ( `121800` paise).

Each saved item in the response looks like this:

```
{
  "product": "65ec39202df3d12ed065251f",
  "product_name": "Chicken Biryani – Example Restaurant, Kolkata",
  "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" },
  "orders": [
    {
      "order": "6ac33f67cd32c8f0c6301fa6",
      "orderid": "T2P-1790000101",
      "line_id": "6ac33f67cd32c8f0c6301fa7",
      "quantity": 2
```

```
    },
    {
      "order": "6ac25d53cd32c8f0c62d7a74",
      "orderid": "T2P-1790000102",
      "line_id": "6ac25d0ccd32c8f0c62d7870",
      "quantity": 3
    }
  ],
  "order_quantity": 5,
  "adjustments": [
    {
      "quantity": -1,
      "orderid": "T2P-1790000102",
      "order": "6ac25d53cd32c8f0c62d7a74",
      "note": "Only 4 available at the shop"
    }
  ],
  "quantity": 4,
  "rate": 25000,
  "weight_kg": 2.2,
  "discount_type": "amount",
  "discount_value": 2000,
  "subtotal": 100000,
  "discount": 2000,
  "taxable": 98000,
  "gst_rate_bp": 500,
  "gst": 4900,
  "total": 102900
}
```

( `gst_rate_bp` is basis points: 500 = 5%. For a percent discount, `discount_value` is also in basis points, e.g. 1250 = 12.5%.)

**Possible errors**

- **409**, an order is already in a purchase: `"T2P-1790000102 already has a food purchase (FP-2627-000001)."`
- **403**, an order is not assigned to this user: `"Order T2P-1790000101 is not assigned to you for pickup."`
- **422**, orders from different cities: field `details.orders`.
- **422**, a quantity change has no reason: field `details.items[0].adjustments[0].note`.
- **422**, the final quantity is below 0: field `details.items[0].adjustments`.

## 6.2 Intercity Logistics ( `INTERCITY_LOGISTICS` )

Cost of sending food from the partner's city to another city.

```
{
  "from_city": "6040eae14a4b6c0008fe1aff",
  "to_city": "6040eb2b4a4b6c0008fe1b01",
```

```
  "rate_per_kg": "38.50",
  "weight_kg": "12.5",
  "reference_no": "AWB 123-4567"
}
```

- `from_city` : **pre-fill with the user's city** ( `meta.me.default_city` ) and lock it. It must be one of `meta.me.cities` .
- `to_city` : from `lookups/cities` ; must be different from `from_city` .
- `reference_no` (AWB / challan number) is optional.
- **Total = rate per kg × weight.** Here 38.50 × 12.5 = ₹481.25 ( `48125` ).

### 6.3 Delivery ( `DELIVERY` )

Records the distance travelled for one order.

1. `GET /v1/expenses/lookups/orders?q=` lists orders where this user is the pickup or delivery partner/boy, from the last 30 days. Each order has a `side` ( `pickup` or `delivery` ) and `sides` , all the sides this user is assigned to. If `sides` has both, let the user choose and send `side` .
2. `GET /v1/expenses/lookups/orders/<orderId>/default-distance?side=delivery` returns the default distance to show (read-only). `side` is only needed when the user is on both sides:
    - **pickup:** office → restaurant;
    - **delivery:** office → customer.

```
{
  "status": "success",
  "data": {
    "order": "6ac36639cd32c8f0c630e184",
    "orderid": "T2P-1790000103",
    "side": "delivery",
    "km": 12.7,
    "source": "google_road",
    "message": null
  }
}
```

`source` is `google_road` (Google Maps) or `straight_line` . If a location is missing, `km` is `null` and `message` explains why, for example "Default distance not available: the restaurant has no location saved.". Still let the user save; show the message.

3. **Request `details` :**

```
{ "order": "6ac36639cd32c8f0c630e184", "actual_distance_km": "14.2", "side": "delivery" }
```

- The server stores `default_distance_km` , `actual_distance_km` and **`deviation_km = actual − default`** (it can be negative).
- **The amount is ₹0 for now**, because the payment rule for delivery isn't decided yet. Delivery expenses therefore never change the cash balance.
- `side` is optional; send it only when the user is on both sides of the order. A side the user is not assigned to gives **403**.

- One delivery expense per order per person per side; a second one gives **409**.

### 6.4 Miscellaneous ( `MISC` )

```
{
  "expense_head": "6ac9...01",
  "description": "2 rolls of packing tape",
  "reason": "Packing boxes for Delhi",
  "amount": "85"
}
```

or, with a head that isn't in the list:

```
{
  "head_other": "Packing tape",
  "description": "2 rolls of packing tape",
  "reason": "Packing boxes for Delhi",
  "amount": "85"
}
```

- Show `meta.expense_heads` plus an **"Other"** option. Other needs `head_other` (free text, up to 100 characters).
- `description` ("Details") and `reason` are required, up to 1,000 characters each.
- `amount` is the total.

### 6.5 Petrol Filling ( `PETROL` )

```
{
  "rate_per_litre": "104.77",
  "litres": "3.25",
  "start_km": "15230",
  "filled_at": "2026-10-10T09:15:00+05:30",
  "vehicle_no": "WB02AB1234",
  "pump_name": "HP, Salt Lake"
}
```

- **Total = rate × litres.** Here 104.77 × 3.25 = ₹340.50 ( `34050` ).
- `start_km` is the odometer reading, required, with up to 1 decimal.
- `filled_at` is optional (defaults to `expense_date` ); `vehicle_no` and `pump_name` are optional.
- **Several petrol entries per day are allowed.**
- If `start_km` is lower than the user's previous petrol entry, the expense still saves but is **flagged** for the office (see `flags` ).

### 6.6 Travelling ( `TRAVEL` )

```
{
  "from_place": "Howrah",
  "to_place": "Sealdah",
```

```
  "travel_mode": "taxi",
  "journey_date": "2026-10-10",
  "purpose": "Collect cash from office",
  "amount": "320"
}
```

- `travel_mode` is one of `meta.travel_modes`. If it is `"other"`, `travel_mode_other` is required (for example `"Ferry"`).
- `journey_date` is optional (defaults to `expense_date`); `purpose` is optional; `amount` is the total.

### 6.7 Cancelled Orders ( `CANCELLED_ORDER` )

Records what happened to the food of an order that was **cancelled** (status `cancel`) or **refused by the customer** ( `rejected_customer` ). There is no money: `amount` is 0 and the cash balance never changes.

**Who sees which cancelled order.** It depends on where the order was just before it was cancelled:

| Order was at (status before cancel) | Shown to |
|---|---|
| Picked up ( `pickup_boy_started` ) | **Pickup city**: the order's pickup partner / pickup boy |
| Received at origin airport ( `delivered_to_cargo_partner` ), cargo loaded to flight ( `cargo_delivery_started` ), cargo off loaded ( `cargo_off_loaded` ), received at destination airport ( `received_destination_airport` ), received by delivery partner ( `delivered_to_delivery_partner` ), delivery boy assigned ( `delivery_boy_assigned` ), out for delivery ( `delivery_boy_started` ) | **Destination city**: the order's delivery partner / delivery boy |
| Anything earlier (vendor approved, pickup boy assigned, …) | Nobody: the food was never picked up |

The server works this out from the order's history; the app only calls the list.

#### Screen flow

1. `GET /v1/expenses/lookups/cancelled-orders?q=` lists the cancelled orders this user has to handle (last 45 days), excluding orders already recorded.

```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac36639cd32c8f0c630e1a1",
      "orderid": "T2P-1790000301",
      "status": "cancel",
      "status_label": "Cancelled",
      "stage_status": "pickup_boy_started",
      "stage_label": "Picked up",
      "side": "pickup",
      "city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "vendor_city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "order_city": { "_id": "6040eb2b4a4b6c0008fe1b01", "name": "DELHI" },
```

```
      "created_date": "2026-10-14T09:20:00.000Z",
      "products": [
        {
          "line_id": "6ac36639cd32c8f0c630e1a2",
          "product": "65ec39202df3d12ed065251f",
          "product_name": "Chicken Biryani – Example Restaurant, Kolkata",
          "quantity": 3,
          "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" }
        }
      ]
    }
  ]
}
```

2. For each product show the quantity (fixed) and let the user add entries. Each entry is one of:

| `type` | Meaning | Fields |
|---|---|---|
| `adjusted` | Adjusted on an order: the food was used for another order | `quantity`, `date`, `orderid` (**required**, an existing order number) |
| `consumed` | Eaten | `quantity`, `date`, `note` (**required**: who consumed it) |
| `scrapped` | Spoiled / thrown away | `quantity`, `date`, `note` (optional: why) |

Show **Left = quantity – all entries** for every product. The total of the entries cannot be more than the quantity. Left may stay above 0.

3. **Request `details`:**

```
{
  "order": "6ac36639cd32c8f0c630e1a1",
  "cancel_reason": "Customer cancelled after pickup",
  "items": [
    {
      "line_id": "6ac36639cd32c8f0c630e1a2",
      "dispositions": [
        {
          "type": "adjusted",
          "quantity": "2",
          "date": "2026-10-15",
          "orderid": "T2P-1790000310"
        },
        {
          "type": "consumed",
          "quantity": "1",
          "date": "2026-10-15",
          "note": "Eaten by office staff"
        }
      ]
    }
```

```
  ]
}
```

| Field | Required | Notes |
|---|---|---|
| `order` | yes | From the list |
| `cancel_reason` | no | Up to 500 characters |
| `items[].line_id` | yes | From `products[].line_id` |
| `items[].dispositions[]` | no | Up to 30 per product |
| `dispositions[].date` | no | `YYYY-MM-DD` ; default today; at most 7 days back |

Each saved item in the response has `quantity` , `dispositions[]` and `left` . For an `adjusted` entry, the server also stores the order id it found, and `product_mismatch: true` if that order does not contain the same product (the record still saves; the office sees a warning). `details.total_left` is the sum of `left` .

4. **Save, edit, approve**
    - Create with `POST /v1/expenses` ( `category: "CANCELLED_ORDER"` ); it is saved as a **draft**. Do not send `submit` .
    - Add more entries later with `PATCH /v1/expenses/:id` , sending all `items` again.
    - When done, call **`POST /v1/expenses/:id/approve`** with body `{}` . The status becomes `approved` and the record can't be edited any more. Approving is allowed even if some items are left; show the user a warning first.
    - `actions.approve` is `true` for the user who handles the order while the record is a draft.

**Errors**

- **422**, `details.order` : the order was cancelled before pickup (nothing to record), or it is not cancelled.
- **403**, the order is handled by the other city's team.
- **409**, a record for this order already exists.
- **422**, `details.items[0].dispositions[1].note` : consumed without "who".
- **422**, `details.items[0].dispositions` : entries are more than the quantity.
- **409**, calling submit on a cancelled-order record.

# 7. Working with expenses

## 7.1 Create: `POST /v1/expenses`

**JSON body:**

```
{
  "category": "PETROL",
  "expense_date": "2026-10-10",
  "notes": "Optional note",
  "client_request_id": "6f1c2b8e-0d1a-4f3e-9a51-3b2c7d9e0a11",
  "submit": true,
```

```
  "details": { "...": "see section 6" }
}
```

| Field | Required | Notes |
|---|---|---|
| `category` | yes | `FOOD_PURCHASE`, `INTERCITY_LOGISTICS`, `DELIVERY`, `MISC`, `PETROL`, `TRAVEL`, `CANCELLED_ORDER` |
| `details` | yes | Category fields (section 6) |
| `expense_date` | no | Default now. No future dates; at most 7 days back |
| `notes` | no | Up to 1,000 characters |
| `client_request_id` | recommended | UUID, one per new expense form (section 3.5) |
| `submit` | no | `true`: send for approval straight away (deducts cash). Otherwise saved as **draft** |

**Multipart (expense + photos in one call):** `Content-Type: multipart/form-data`

- field **`data`**: the JSON body above, as a string;
- field **`files`**: up to 5 files (repeat the field for each file).

Response **201**, `data` = the full expense (section 7.2). Possible extra fields:

- `warning`: e.g. default distance not available;
- on create + submit: `balance_after` (paise) and a `warning` if the balance went negative.

**Documents are optional** in every category.

## 7.2 The expense object: `GET /v1/expenses/:id`

```
{
  "status": "success",
  "data": {
    "expense": {
      "_id": "6ac990c0195df90f1c7635b4",
      "expense_no": "FP-2627-000001",
      "category": "FOOD_PURCHASE",
      "status": "submitted",
      "expense_date": "2026-10-10T06:41:28.219Z",
      "business_date": "2026-10-10",
      "source": "logistics_app",
      "amount": { "subtotal": 118000, "discount": 2000, "tax": 5800, "total": 121800 },
      "details": {
        "orders": [ "..." ],
        "vendor_city": { "_id": "...", "name": "KOLKATA" },
        "items": [ "..." ],
        "total_weight_kg": 3.2
      },
      "order_refs": [
        { "order": "6ac33f67cd32c8f0c6301fa6", "orderid": "T2P-1790000101" },
        { "order": "6ac25d53cd32c8f0c62d7a74", "orderid": "T2P-1790000102" }
      ],
```

```
      "notes": "Bought at the counter",
      "documents": [
        {
          "_id": "6ac990c0195df90f1c7635be",
          "name": "bill.jpg",
          "mime": "image/jpeg",
          "size": 184220,
          "uploaded_at": "2026-10-10T06:41:30.303Z",
          "removed": false
        }
      ],
      "expense_for": {
        "_id": "605dc565d5eaba0008a50e4e",
        "name": "Ravi Kumar",
        "mobile": "9800000010",
        "user_type": "delivery_partner"
      },
      "submitted_by": { "...": "same shape" },
      "city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "office": { "_id": "60ef0e96d4473c18f88872f5", "name": "KOLKATA OFFICE" },
      "review": {
        "reviewed_by": null,
        "reviewed_at": null,
        "approval_note": "",
        "rejection_reason": ""
      },
      "ledger": { "state": "posted", "cycle": 1 },
      "flags": [],
      "submitted_at": "2026-10-10T06:41:31.374Z",
      "cancel_reason": "",
      "version": 3,
      "createdAt": "2026-10-10T06:41:28.257Z",
      "updatedAt": "2026-10-10T06:41:31.374Z"
    },
    "category_label": "Food Purchase",
    "actions": {
      "edit": false,
      "upload": false,
      "submit": false,
      "recall": true,
      "approve": false,
      "reject": false,
      "cancel": true
    },
    "history": [
      {
        "action": "created",
        "from_status": null,
        "to_status": "draft",
```

```
        "actor": { "name": "Ravi Kumar", "...": "..." },
        "source": "logistics_app",
        "note": "",
        "at": "2026-10-10T06:41:28.257Z"
      },
      { "action": "submitted", "from_status": "draft", "to_status": "submitted", "...": "..." }
    ],
    "ledger_entries": [
      {
        "txn_no": "CL-2627-000001",
        "type": "EXPENSE_DEBIT",
        "direction": "debit",
        "amount": 121800,
        "running_balance": 378200,
        "business_date": "2026-10-10"
      }
    ],
    "cash_balance": 378200
  }
}
```


Key fields for the UI:

| Field | Use |
|---|---|
| `expense_no` | Show as the reference (FP-…, IL-…, DL-…, MS-…, PF-…, TR-…) |
| `status` | Status badge |
| `amount.total` | Amount in paise |
| `review.rejection_reason` | Show when `status` is `rejected` |
| `cancel_reason` | Show when `status` is `cancelled` |
| `actions` | Which buttons to show (section 4.1) |
| `history[]` | Timeline: created, submitted, rejected, resubmitted, … |
| `ledger.state` | `none` (not deducted), `posted` (deducted), `reversed` (put back). Rarely `pending` or `reversal_pending` for a moment while the server finishes the entry |
| `flags[]` | Warnings the office will see, e.g. possible duplicate. Optional to show |
| `cash_balance` | The user's cash balance now |

## 7.3 My expenses list: `GET /v1/expenses`

**Query parameters (all optional):**

| Param | Example | Notes |
|---|---|---|
| `page` | `1` | Starts at 1 |
| `limit` | `20` | Maximum 100 |

| Param | Example | Notes |
|---|---|---|
| `status` | `submitted` or `draft,rejected` | Comma-separated |
| `category` | `PETROL` | Comma-separated |
| `from` , `to` | `2026-10-01` | By expense date (YYYY-MM-DD) |
| `q` | `T2P-1790000101` | Search expense number, order number, note |
| `sort` | `expense_date` | Also `amount` , `created` , `expense_no` , `status` |
| `dir` | `desc` | Or `asc` |


```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac990c0195df90f1c763660",
      "expense_no": "PF-2627-000001",
      "category": "PETROL",
      "category_label": "Petrol Filling",
      "status": "submitted",
      "expense_date": "2026-10-10T03:45:00.000Z",
      "amount": { "subtotal": 34050, "discount": 0, "tax": 0, "total": 34050 },
      "details": { "litres": 3.25 },
      "order_refs": [],
      "document_count": 1,
      "review": { "rejection_reason": "" },
      "version": 2
    }
  ],
  "totals": {
    "count": 6,
    "amount": 194625,
    "by_status": {
      "draft": { "count": 0, "amount": 0 },
      "submitted": { "count": 5, "amount": 146500 },
      "approved": { "count": 0, "amount": 0 },
      "rejected": { "count": 0, "amount": 0 },
      "cancelled": { "count": 1, "amount": 48125 }
    }
  },
  "pagination": { "page": 1, "limit": 20, "total": 6, "pages": 1 }
}
```


`totals.by_status` gives the counts for the status tabs. The list rows contain only summary fields; open `GET /v1/expenses/:id` for full details.

### 7.4 Edit: `PATCH /v1/expenses/:id`

Only allowed while the status is **draft** or **rejected** ( `actions.edit` is true).

```
{
  "version": 4,
  "expense_date": "2026-10-09",
  "notes": "Corrected amount",
  "details": {
    "head_other": "Packing tape",
    "description": "2 rolls of tape",
    "reason": "Packing boxes for Delhi",
    "amount": "80"
  }
}
```

- **Send the complete `details` again**, not only the changed field.
- `expense_date` and `notes` are optional.
- **Food Purchase:**
  - send `items` for every line;
  - send `orders` only if the selection changed (if you leave `orders` out, the current orders are kept).
- Editing does **not** submit. After editing a rejected expense, call submit (7.5).

### 7.5 Submit, move to draft, cancel

| Call | Body | When |
|---|---|---|
| `POST /v1/expenses/:id/submit` | `{}` or `{ "version": 4 }` | Draft or rejected. Deducts cash. Response has `balance_after` and maybe `warning` |
| `POST /v1/expenses/:id/recall` | `{}` | Submitted, before the office reviews it. Back to draft; cash is put back |
| `POST /v1/expenses/:id/cancel` | `{ "reason": "Entered twice" }` | Draft (reason optional), submitted or rejected (reason **required**, at least 3 characters). Cash is put back if it was deducted |

Each returns the full expense as `data` (same as 7.2), plus `message`.

```
{
  "status": "success",
  "message": "Expense submitted",
  "data": { "...": "..." },
  "ledger_entry": "CL-2627-000001",
  "balance_after": 378200
}
```

```
{
  "status": "success",
  "message": "Expense submitted",
  "data": { "...": "..." },
  "ledger_entry": "CL-2627-000009",
  "balance_after": -72450,
```

```
  "warning": "Cash balance is now negative for this person."
}
```

### 7.6 Photos and documents

| Call | Notes |
|---|---|
| `POST /v1/expenses/:id/documents` | Multipart, field **`files`** (1–5 files). Only while draft or rejected. Max 5 attached per expense |
| `DELETE /v1/expenses/:id/documents/:docId` | Removes a document while draft or rejected |
| `GET /v1/expenses/:id/documents/:docId` | Returns the file. Add `?download=1` to download instead of view |

- **Allowed:** JPG, PNG, WEBP, HEIC, PDF, up to 10 MB each. The type is checked from the file content, not the name.
- **Compression:** the server compresses photos (longest side 1600 px, JPEG) and removes photo metadata, including GPS, so upload the camera photo as it is. HEIC and PDF are stored unchanged. The stored file may therefore be `.jpg` even if you uploaded `.png`.
- **Viewing:** files are **private**. Load them with the `Authorization` header, e.g. an image loader with headers, or download to a temporary file. There are no public links.

### 7.7 Duplicate protection the app may see

| Situation | Response |
|---|---|
| Same `client_request_id` sent again | 200, the same expense |
| Order already in another active food purchase | 409, with the order number and the FP number |
| Second delivery expense for the same order and side | 409 |
| Same photo already attached to another expense, or same amount on the same day | Saved, but `flags[]` warns the office |

## 8. My cash: `GET /v1/cash-ledger/me`

**Query (optional):**

| Param | Notes |
|---|---|
| `from`, `to` | `YYYY-MM-DD`, the period for the summary and entries |
| `type` | One of the entry types below |
| `page` | Default 1 |
| `limit` | Default 50, maximum 200 |

```
{
  "status": "success",
  "data": {
```

```
  "user": {
    "_id": "605dc565d5eaba0008a50e4e",
    "name": "Ravi Kumar",
    "mobile": "9800000010",
    "user_type": "delivery_partner"
  },
  "account": {
    "balance": 378200,
    "opening_set": true,
    "city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" }
  },
  "summary": {
    "opening_balance": 0,
    "received_from_head_office": 500000,
    "expenses_deducted": 121800,
    "closing_balance": 378200,
    "by_type": {
      "OPENING_BALANCE": { "amount": 0, "count": 0 },
      "CASH_RECEIVED_HO": { "amount": 500000, "count": 1 },
      "EXPENSE_DEBIT": { "amount": 121800, "count": 1 },
      "...": "..."
    }
  },
  "entries": [
    {
      "txn_no": "CL-2627-000002",
      "type": "EXPENSE_DEBIT",
      "type_label": "Expense",
      "direction": "debit",
      "amount": 121800,
      "running_balance": 378200,
      "entry_date": "2026-10-10T06:41:28.219Z",
      "business_date": "2026-10-10",
      "expense": "6ac990c0195df90f1c7635b4",
      "expense_no": "FP-2627-000001",
      "expense_status": "submitted",
      "note": "Food Purchase FP-2627-000001"
    },
    {
      "txn_no": "CL-2627-000001",
      "type": "CASH_RECEIVED_HO",
      "type_label": "Cash Received from Head Office",
      "direction": "credit",
      "amount": 500000,
      "running_balance": 500000,
      "business_date": "2026-10-09",
      "mode": "upi",
      "reference": "UTR 4521889",
      "note": ""
```

```
      }
    ],
    "pagination": { "page": 1, "limit": 50, "total": 2, "pages": 1 }
  }
}
```

**How to show it**

- `account.balance` is the **cash in hand now**. Show it in red if negative (the person has spent more than they received).
- Entries are newest first. `direction`: credit = money in (+), debit = money out (–). `running_balance` is the balance after that entry.
- For `EXPENSE_DEBIT` entries, show `expense_no` with a badge from `expense_status` (Pending approval or Approved), and tap to open the expense.
- For `EXPENSE_REVERSAL`, `note` says why the money came back, e.g. `"MS-2627-000001 Rejected: Bill photo missing"`.

| Entry type | Meaning | Direction |
|---|---|---|
| `OPENING_BALANCE` | Starting cash, entered by the office | credit |
| `CASH_RECEIVED_HO` | Cash sent by Head Office (`mode`: cash/bank/upi/other, `reference`) | credit |
| `EXPENSE_DEBIT` | An expense was submitted | debit |
| `EXPENSE_REVERSAL` | An expense was rejected, cancelled or moved back to draft | credit |
| `ADJUSTMENT_CREDIT` / `ADJUSTMENT_DEBIT` | Correction by the office, with a reason in `note` | credit / debit |

Ledger entries are never edited or deleted; corrections are always new entries.

## 9. Push notifications

When the office **approves**, **rejects**, or **cancels an approved** expense, the server sends a Firebase push to the user's `device_token`. The app already registers that token with `POST /app/update-token`.

```
{
  "notification": {
    "title": "Expense rejected",
    "body": "Miscellaneous MS-2627-000001 ₹85.00 — Bill photo missing"
  },
  "data": { "type": "expense", "expense_id": "6ac990c0195df90f1c763700", "status": "rejected" }
}
```

On tap, open `GET /v1/expenses/{expense_id}`. Titles: "Expense approved", "Expense rejected", "Approved expense cancelled".

## 10. Endpoint reference (app)

| Method | Path | Purpose |
|---|---|---|
| POST | `/app/logistic-login` | Request OTP (existing) |
| POST | `/app/verify-logistic` | Verify OTP, get `token` |
| GET | `/v1/expenses/meta` | Categories, form settings, my profile and cities |
| GET | `/v1/expenses/lookups/cities` | Cities |
| GET | `/v1/expenses/lookups/expense-heads` | Miscellaneous heads |
| GET | `/v1/expenses/lookups/food-purchase-orders?q=` | My orders waiting for a food purchase |
| GET | `/v1/expenses/lookups/food-purchase-lines?orders=id,id` | Combined product lines for ticked orders |
| GET | `/v1/expenses/lookups/orders?q=` | My orders for a delivery expense |
| GET | `/v1/expenses/lookups/orders/:id/default-distance?side=` | Default distance for an order |
| GET | `/v1/expenses/lookups/cancelled-orders?q=` | Cancelled orders I have to record |
| POST | `/v1/expenses` | Create (draft or submit), JSON or multipart |
| GET | `/v1/expenses` | My expenses (filters, totals, pages) |
| GET | `/v1/expenses/:id` | Expense details |
| PATCH | `/v1/expenses/:id` | Edit a draft or rejected expense |
| POST | `/v1/expenses/:id/submit` | Submit or resubmit |
| POST | `/v1/expenses/:id/recall` | Move back to draft |
| POST | `/v1/expenses/:id/cancel` | Cancel |
| POST | `/v1/expenses/:id/approve` | **Cancelled Orders only:** approve my record (final) |
| POST | `/v1/expenses/:id/documents` | Add photos/files |
| DELETE | `/v1/expenses/:id/documents/:docId` | Remove a photo/file |
| GET | `/v1/expenses/:id/documents/:docId` | View/download a photo/file |
| GET | `/v1/cash-ledger/me` | My cash balance and entries |

Office-only endpoints return **403** for app users: approve (except Cancelled Orders), reject, export, dashboard, other people's ledgers, Head Office cash, adjustments.

## 11. Testing checklist

Please check these before release:

1. **Login:** OTP login stores the token; an expired or removed token sends the user back to login (401).

2. **Food Purchase:**
    - two orders with the same product show one line with the summed quantity;
    - the order quantity can't be typed;
    - a quantity change without a reason shows the field error;
    - every product needs a rate unless its final quantity is 0;
    - the totals shown after saving match the server.
3. **Food Purchase, orders:** using an order that is already in another purchase shows the 409 message; orders from two cities show an error.
4. **Intercity:** From City is pre-filled and locked.
5. **Delivery:** the default distance is shown read-only, and "not available" messages are displayed.
6. **Petrol:** two fillings on the same day both save.
7. **Travel:** "Other" requires the mode name.
8. **Retries:** turn off the network while submitting, turn it on and retry; you get the same expense, not a duplicate ( `client_request_id` ).
9. **Rejection:**
    - the rejected expense shows the reason;
    - edit → submit works;
    - the history shows "resubmitted";
    - the cash went back on rejection and was deducted again on resubmit.
10. **Cancel and move to draft:** the cash balance goes back up.
11. **Photos:** upload from the camera (large photo) and from the gallery, and a PDF; view them with the token; remove works only in draft or rejected.
12. **My cash:** balance, period summary and entries (newest first, with running balance); a negative balance is shown clearly.
13. **Push:** approve and reject from the admin panel; the notification opens the expense.
14. **Errors:** 422 errors appear next to the right fields; 409 reloads the expense; 429 waits and retries.
15. **Cancelled Orders:**
    - a pickup partner sees orders cancelled after pickup; a delivery partner sees orders cancelled from the origin airport onwards;
    - adjusted needs an order number, consumed needs "who";
    - Left updates as entries are added;
    - approve makes it final, with no edit afterwards;
    - the order disappears from the list once recorded.
16. **Delivery, partner on both sides:** the user can choose pickup or delivery.

## 12. Current limitations

- **Delivery amount:** delivery expenses are saved with ₹0 until T2P decides the payment rule. The field will appear in `amount` later, with no app change needed.
- **Pickup default distance:** restaurant locations are not saved yet, so the default distance for **pickup** trips usually returns `km: null` with a message. Delivery-side distances work.
- **HEIC:** iPhone HEIC photos are stored without compression. To save data, the app may convert to JPEG before uploading (optional).

- **Approvals:** expenses are approved only in the admin panel, except Cancelled Orders, which the partner approves in the app.
- **Cancelled-order history:** who sees a cancelled order is worked out from the order's status history. If a status change was never recorded, the order may not appear in the list; report such orders to the T2P office.

For questions about this API, contact the T2P tech team.